

CommonGround — System Architecture

Version 1.0 · Hack4Change 2026 · GMHSC Donation Coordination Platform · June 2026

Contents

1. Executive Summary
2. High-Level Architecture
3. Technology Stack
4. Project Structure
5. User Roles & Entry Points
6. Backend Architecture
7. Frontend Architecture
8. API Surface
9. Authentication & Security
10. Real-Time Layer
11. Donation Matching Engine
12. Database Overview
13. How Items Are Stored
14. Domain Workflows
15. Deployment & Operations
16. Testing & Scripts

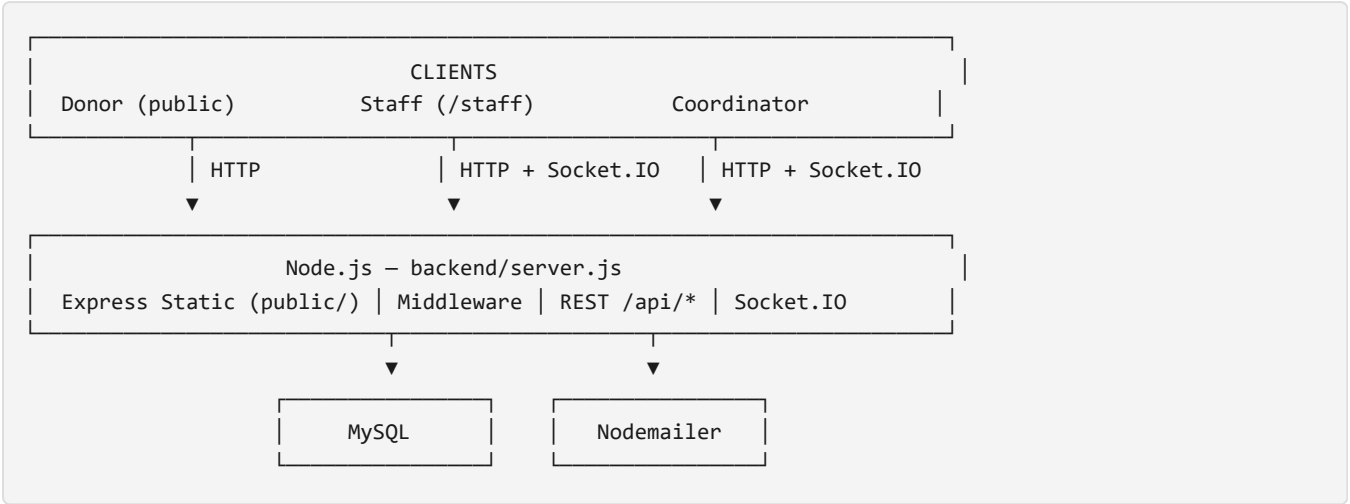
1. Executive Summary

CommonGround is a real-time donation coordination platform for the Greater Moncton Homelessness Steering Committee (GMHSC) network of 28 homeless-serving organizations. It connects donors, shelter staff, and network coordinators in a single web application.

- **Donors** — Browse live needs and submit donations without an account.
- **Shelter staff** — Manage inventory, post shortages, confirm donations, request surplus, chat.
- **Coordinators** — Network-wide KPIs, org/staff management, transfers, exports.

The app runs as a **monolithic Node.js process**: Express serves static HTML/JS from `public/` and REST API under `/api`, backed by **MySQL**, with **Socket.IO** for live updates. No frontend build step, no microservices, no ORM.

2. High-Level Architecture



3. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, vanilla JavaScript (no bundler)
Backend	Node.js 18+, Express.js
Database	MySQL via mysql2 connection pool
Real-time	Socket.IO 4.x
Auth	JWT (access + refresh) in httpOnly cookies
Email	Nodemailer
Matching	Rule-based scoring (no external AI API)
Security	Helmet, rate-limit, express-validator, bcryptjs
Tests	Jest + Supertest
Deploy	Render + Aiven MySQL (SSL)

4. Project Structure

```
commonGround/
├── backend/
│   ├── server.js           # Express + Socket.IO + static routes
│   ├── config/database.js  # mysql2 pool
│   ├── controllers/        # auth.controller.js
│   ├── middleware/         # auth, security
│   ├── models/             # One model per table
│   ├── routes/             # public, auth, staff, coordinator, ai
│   └── utils/              # email, jwt, hash, logger
├── public/                 # Frontend HTML, CSS, JS
├── scripts/                # migrate.js, run-seed-demo.js
├── tests/                  # Jest suites
├── seed-demo.sql
├── render.yaml
└── package.json
```

5. User Roles & Entry Points

User	URLs	Auth
Donor	/, /give, /donate, /all-needs	None
Staff	/login → /staff	JWT (org-scoped)
Coordinator	/login → /coordinator	JWT + isAdmin

Staff dashboard sections

Dashboard, Inventory, Your Needs, Other Needs, Donations, Analytics, Chats.

Coordinator dashboard sections

Overview, Organizations, Staff, All Needs, Donations, Inventory, Transfers, Chat, Analytics.

6. Backend Architecture

Models (data access)

Model	Table
organization.model.js	organizations
staff.model.js	staff_members
admin.model.js	admins
inventory.model.js	inventory_items
needs.model.js	needs
donation.model.js	donations
surplusRequest.model.js	surplus_requests
surplusTransfer.model.js	surplus_transfers
chatThread / chatMessage	chat_threads, chat_messages

Route authorization

Prefix	Access
/api/public/*	No auth
/api/auth/*	Login, cookies
/api/staff/*	authenticateToken
/api/coordinator/*	authenticateToken + requireAdmin
/api/ai/match-donation	Public (rate-limited)
/api/ai/network-insights	Admin only

7. Frontend Architecture

Multi-page app with two large dashboard SPAs (`staff.html`, `coordinator.html`). Shared client code in `public/js/api.js` (fetch + cookies), `i18n.js` (EN/FR), theme utilities. Dashboards use sidebar section switching without full page reloads.

8. API Surface

Public

- GET `/api/public/orgs-with-needs`, `/needs`, `/organizations`
- POST `/api/public/donations`
- PATCH `/api/public/donations/:id/match`

Auth

- POST `/api/auth/login`, `logout`, `refresh`
- GET `/api/auth/me`

Staff (JWT)

- Inventory, needs, donations CRUD + confirm
- Surplus requests, transfers, expiring items, chat

Coordinator (JWT + admin)

- Overview, orgs, staff, needs, donations pipeline
- Surplus, transfers, analytics, CSV exports

Health

- GET `/api/health` — GET `/api/health?deep=true` (MySQL ping)

9. Authentication & Security

Login verifies bcrypt password against `staff_members` or `admins`. JWT access + refresh tokens stored in **httpOnly cookies**. Coordinator tokens include `isAdmin: true`.

Control	Implementation
Headers	Helmet + CSP
CORS	CORS_ORIGINS env var
Rate limit	200 req / 15 min (production)
Input	Sanitization middleware
SQL	Parameterized queries + injection blocking

10. Real-Time Layer (Socket.IO)

Socket.IO shares the HTTP server. Events: `donation:new`, `need:new`, `need:fulfilled`, `chat:newMessage`.

11. Donation Matching Engine

POST `/api/ai/match-donation` — rule-based, no external LLM.

Factor	Points
Category match	+10
Item name similarity	+15
Urgency (critical→low) × 3	up to +12
Quantity ≥ need	+5

Returns **top 3** organizations from all unfulfilled needs network-wide.

12. Database Overview

```
organizations —< staff_members
organizations —< inventory_items
organizations —< needs
organizations —< donations (preferred_org_id, matched_org_id)
inventory_items —< surplus_requests, surplus_transfers
chat_threads —< chat_messages
admins — network coordinators
```

Table	Purpose
organizations	28 network orgs
staff_members	Org-scoped users
inventory_items	Per-org stock
needs	Per-org shortages
donations	Donor pipeline
surplus_requests	Staff surplus requests
surplus_transfers	Coordinator stock moves
chat_threads / chat_messages	In-app messaging

Schema: `npm run migrate` (destructive reset). **Seed:** `npm run seed:demo`.

13. How Items Are Stored

No global product catalog. Items are org-owned rows with free-text `item_name` (e.g. "Shelf-Stable Milk (1L)").

Concept	Table	Uniqueness
Stock	inventory_items	One row per (org + name + category)
Shortage	needs	One open need per (org + name + category)
Donation	donations	Standalone until matched to one org

If all 28 orgs have milk

- **Inventory:** 28 separate rows — independent quantity, status, expiry.
- **Needs:** Up to 28 need rows; matcher returns top 3 by score.
- **Transfers:** Reference specific `inventory_item_id`, not global "milk".
- **Confirm donation:** Updates only the matched org's inventory row.

Status logic: quantity < target → low; quantity > target → surplus; else available.

14. Domain Workflows

Donation lifecycle

Donor submits → POST /api/public/donations (pending)
→ confirmation email → Socket.IO donation:new
→ POST /api/ai/match-donation → PATCH match (matched)
→ staff confirm → inventory++ → thank-you email
→ coordinator may set delivered

Surplus redistribution

Staff flags surplus → coordinator sees network surplus
Staff B posts need → request surplus OR coordinator initiates transfer
Coordinator approves → transfer tracked → receiving staff completes

15. Deployment & Operations

npm install → cp .env.example .env → npm run setup → npm run dev
Production: Render (render.yaml), npm start, /api/health
MySQL: Aiven with DB_SSL_ENABLED=true, ca.pem

16. Testing & Scripts

Script	Purpose
npm run setup	install + migrate + seed
npm run dev	nodemon development
npm start	production server
npm test	Jest test suites

Appendix: Design Principles

1. Monolith simplicity — one process, one deploy.
2. Org-scoped data — inventory and needs belong to organizations.
3. Free-text items — flexible naming; fuzzy matching on submit.
4. Progressive auth — donors need no account.
5. Real-time dashboards via Socket.IO.
6. Rule-based matching — no paid AI APIs required.